

西南交通大学
Southwest Jiaotong University

虚拟化技术与Docker

CLOUD COMPUTING

丛荣

西南交通大学
计算机与人工智能学院

课程回顾：资源池

云计算的定义：

- 根据NIST (美国国家标准与技术研究院) 定义：
- 云计算是一种计算模型，允许无处不在的、便捷的、按需的网络访问
- 具备**共享的可配置计算资源池**（包括网络、服务器、存储、应用和服务）
- 资源能够用最小的管理代价或服务提供商的交互实现快速配置和发布

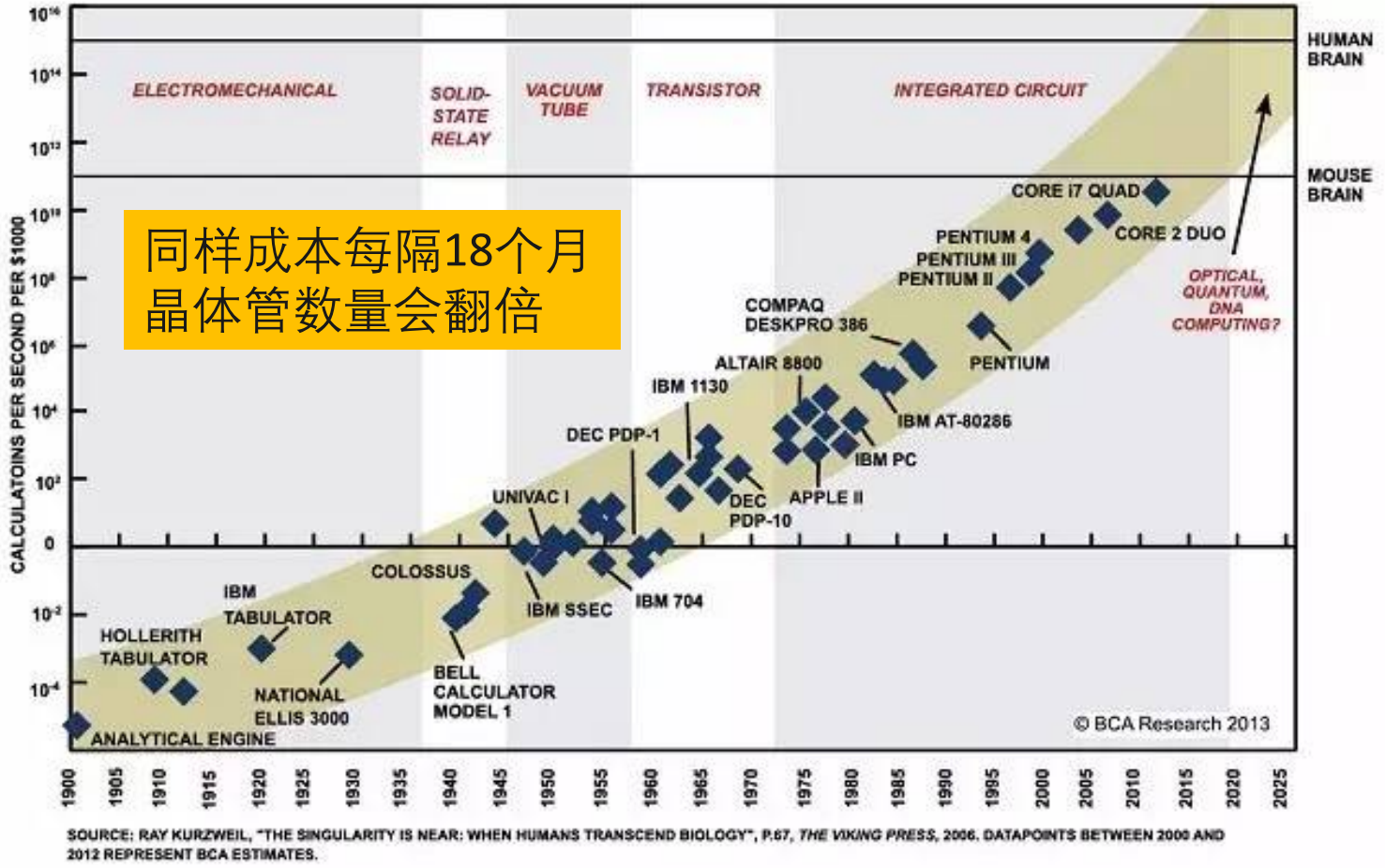
怎么将一台物理机切分给多个用户？

NIST
National Institute of Standards and Technology

"Cloud computing is a model for enabling convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction. "

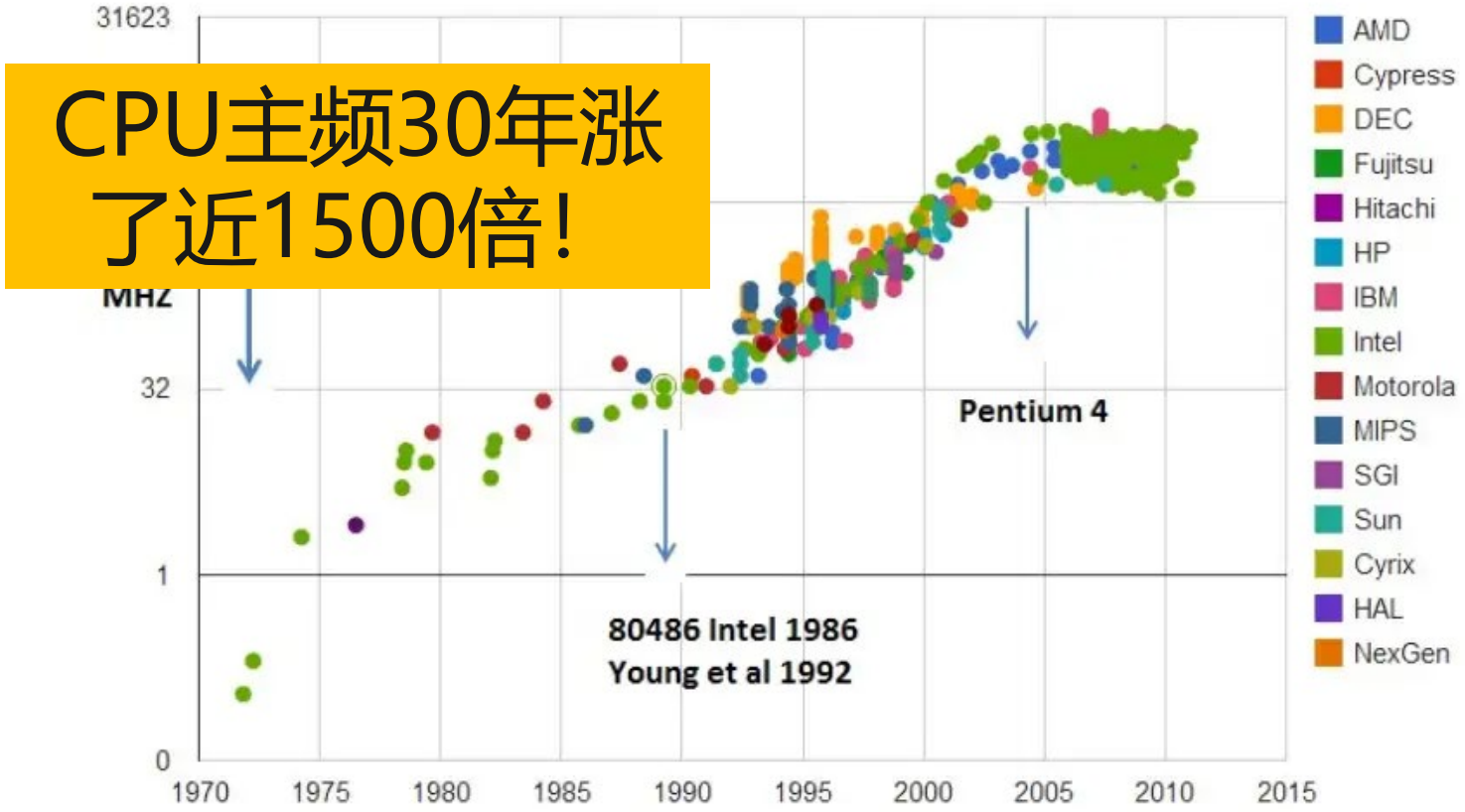
虚拟化的产生动机：硬件性能过剩

□硬件发展趋势： Moore’s Law



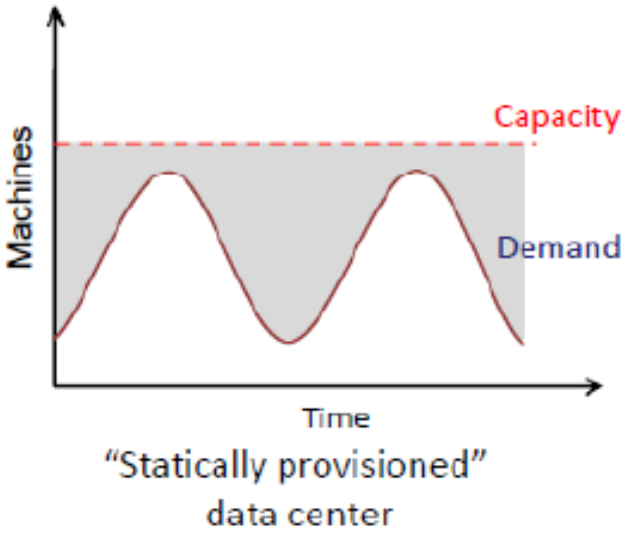
虚拟化的产生动机：硬件性能过剩

□硬件发展趋势：CPU主频



虚拟化的产生动机：数据中心的痛点

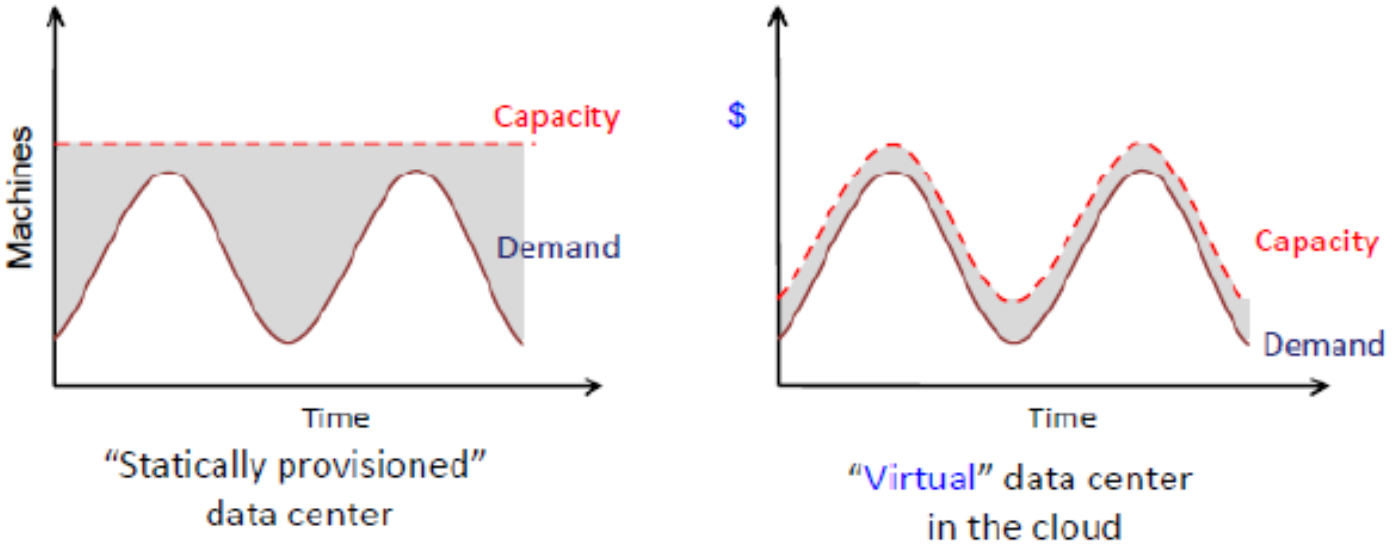
- 一台普通PC服务器的 CPU 负载通常只有 5%-15%
- 大量电力、场地和维护成本被闲置硬件消耗



极低的资源利用率←→高昂运营成本

虚拟化的产生动机：数据中心的痛点

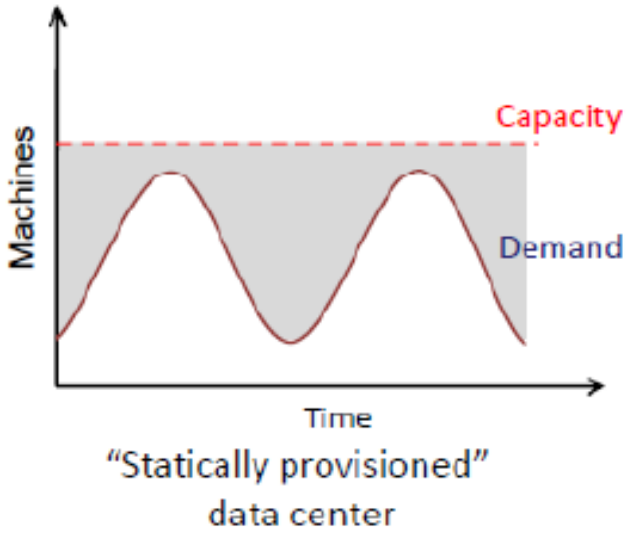
- 一台普通PC服务器的 CPU 负载通常只有 5%-15%
- 大量电力、场地和维护成本被闲置硬件消耗



极低的资源利用率←→高昂运营成本

虚拟化的产生动机：数据中心的痛点

- 一台普通PC服务器的 CPU 负载通常只有 5%-15%
- 大量电力、场地和维护成本被闲置硬件消耗

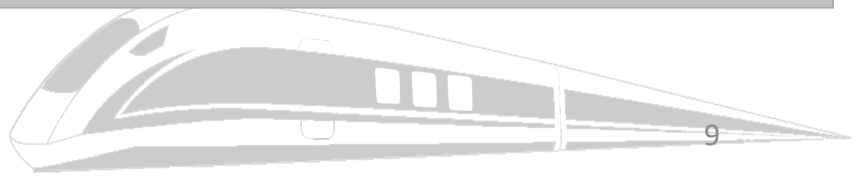


极低的资源利用率←→高昂运营成本

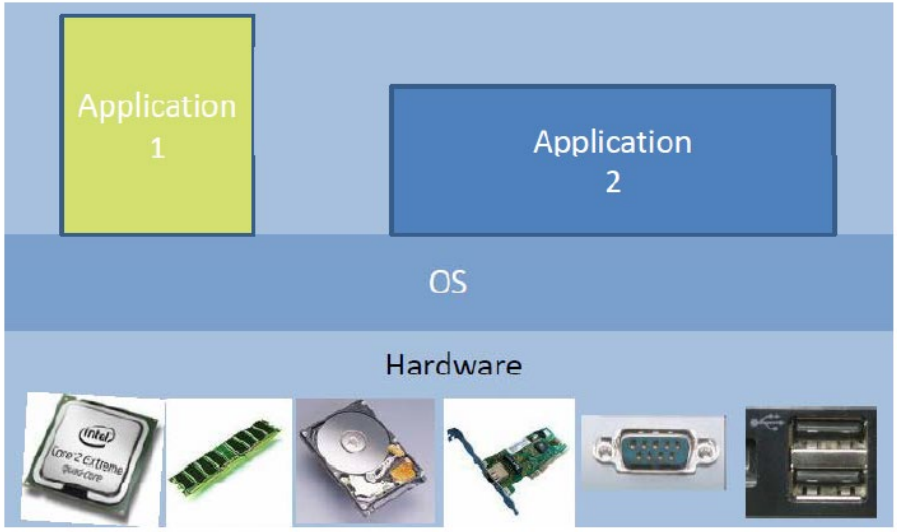
虚拟化的定义

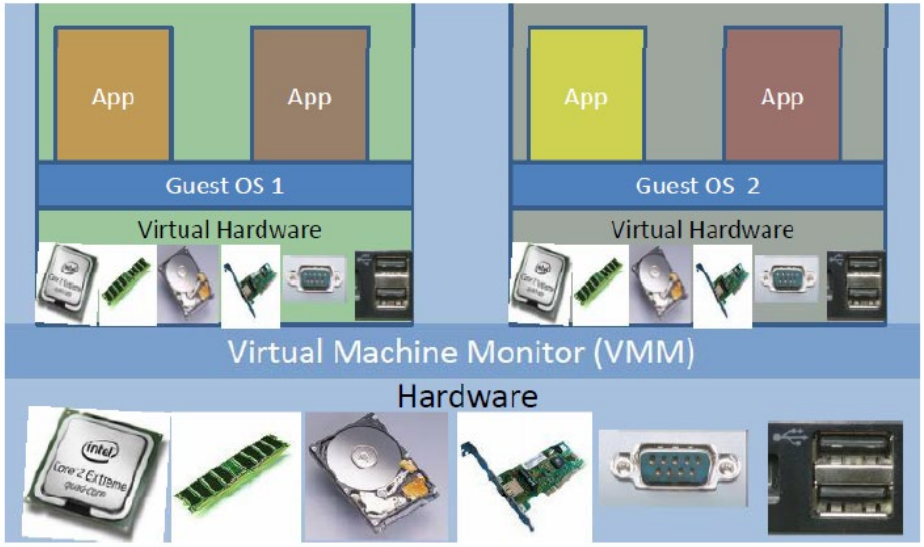
- 虚拟化就是Virtualization
- 虚拟化是一种资源管理技术
- 实体资源：CPU、内存、磁盘、网络...
- 虚拟化：让每个进程看起来拥有单独的硬件资源
- 虚拟机：由虚拟化层提供的高效独立的虚拟计算机系统
- 从软件角度来看，虚拟机=真实机器

虚拟化（Virtualization）是一种资源管理技术，是将计算机的各种实体资源（CPU、内存、磁盘空间、网络适配器等），予以抽象、转换后呈现出来并可供分割、组合为一个或多个电脑配置环境。由此，打破实体结构间的不可切割的障碍，使用户可以比原本的配置更好的方式来应用这些电脑硬件资源。这些资源的新虚拟部分是不受现有资源的架设方式，地域或物理配置所限制。



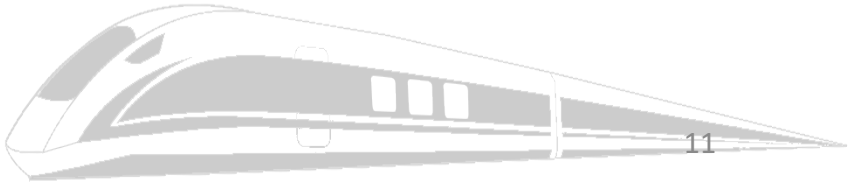
虚拟化的定义


Application 1
Application 2
OS
Hardware


App
App
App
App
Guest OS 1
Guest OS 2
Virtual Hardware
Virtual Hardware
Virtual Machine Monitor (VMM)
Hardware

虚拟化带来的优势

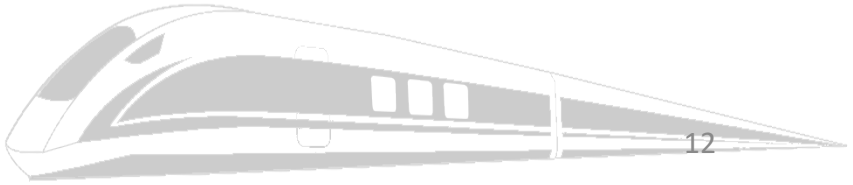
- 资源利用率：资源动态分配和调度
- 运维成本：自动化开通能力
- 升级迁移：方便系统的平滑升级与跨硬件迁移
- 测试生产：安全的隔离环境



硬件虚拟化V.S.软件虚拟化

- 虚拟化是让每个进程看起来拥有单独的硬件资源

如何实现虚拟化（怎么骗过OS）？



硬件虚拟化V.S.软件虚拟化

如何实现虚拟化（怎么骗过OS）？

- 软件虚拟化
  - 通过虚拟机监控器 (VMM) 捕获并翻译特权指令
  - 兼容性好
  - 系统性能开销较大，实现逻辑复杂
  - KEMU、VMware





硬件虚拟化V.S.软件虚拟化

如何实现虚拟化（怎么骗过OS）？

- 硬件虚拟化
  - 依靠 CPU 等底层硬件架构级的扩展支持，增加一套专门给虚拟机的特权指令通道（Ring-1）
  - 无需软件干预，性能极高
  - 和硬件设计绑定
  - Intel VT-x, AMD-v







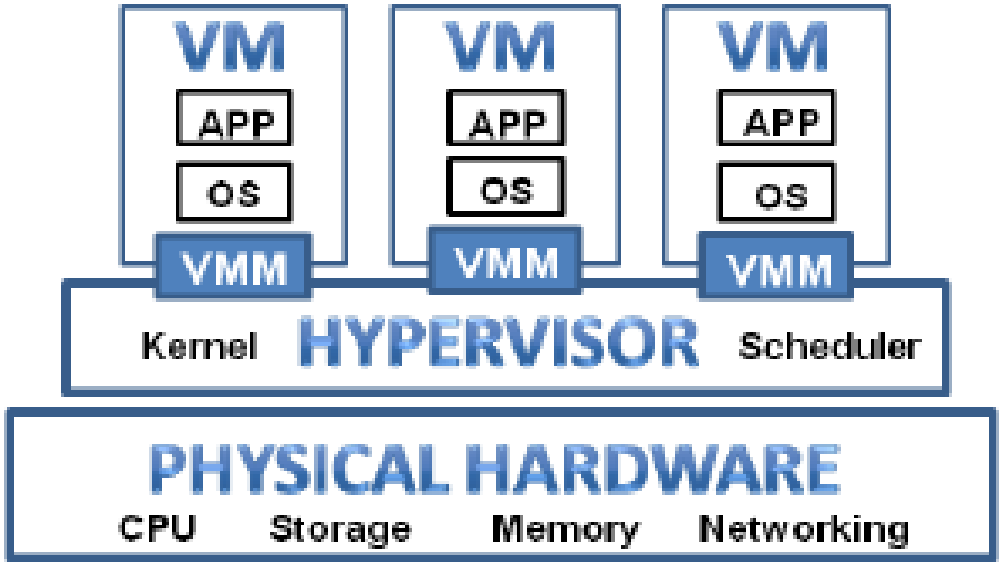
虚拟化的发展历程

- **1.0 时代（IBM大型机）**
  IBM 提出虚拟化概念（System/360），解决大型机多用户分时共享
- **2.0 时代（VMware）**
  VMware 提出全虚拟化方案，解决了 x86 架构不适合虚拟化的缺陷
- **3.0 时代（Xen）**
  剑桥大学Xen 项目开源，提出半虚拟化，修改 Guest OS 执行特权指令
- **4.0 时代（KVM）**
  KVM 深度集成于 Linux 内核，企业公有云 IaaS标准



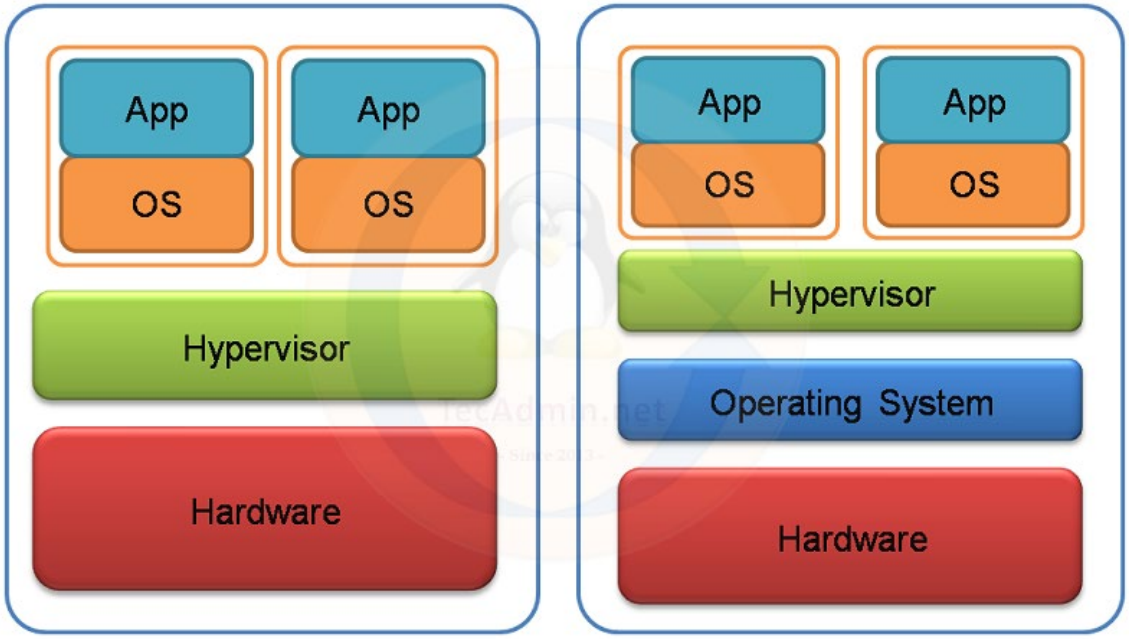
虚拟化核心组件：Hypervisor/VMM

- 职责1：物理硬件资源→虚拟资源的逻辑映射
- 职责2：虚拟机的上下文隔离，保障多租户环境安全
- 运行虚拟机时，Hypervisor 中的 VM kernel 会装载 VMM



Hypervisor典型架构

- 裸机虚拟化架构 (Bare-metal Virtualization)
- 寄居虚拟化架构 (Hosted Virtualization)



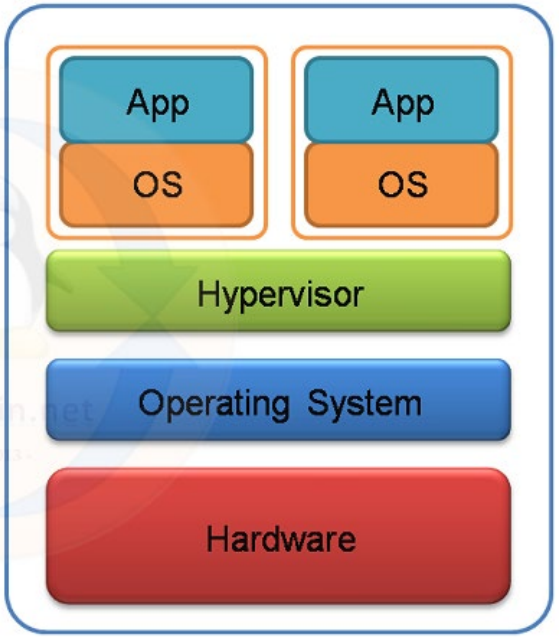
Type 1 (Bare Metal) Virtualization

Type 2 (Hosted) Virtualization

Hypervisor典型架构

- 寄居虚拟化架构 (Hosted Virtualization)
  - Hypervisor 作为普通进程运行在Host OS之上
  - 例如：Vmware（个人PC）
  - 硬件兼容性好、部署方便
  - 系统开销大

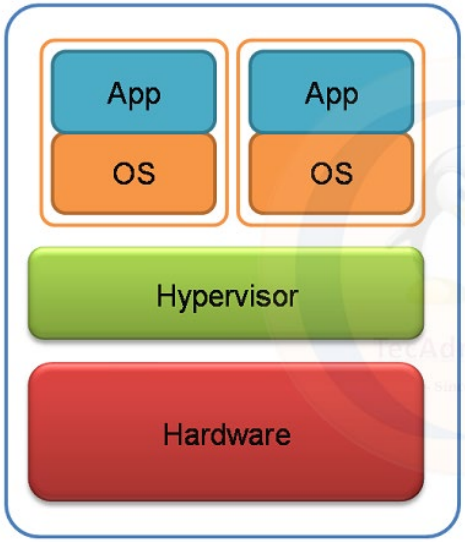
    I/O 请求需经过 Host OS 转发，上下文切换频繁



Type 2 (Hosted) Virtualization

Hypervisor典型架构

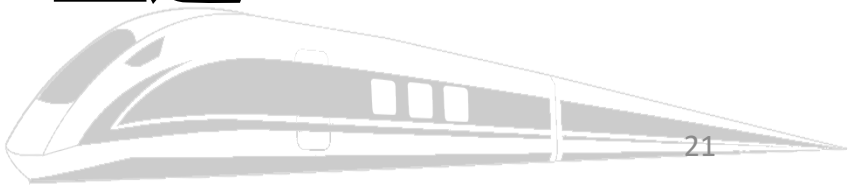
- 裸机虚拟化架构 (Bare-metal Virtualization)
  - Hypervisor 直接运行于裸机硬件之上
  - 实现从虚拟资源到物理资源的映射
  - 例如：KVM（数据中心）
  - 省去底层 Host OS 的中间损耗
  - 系统性能最接近物理机
  - 硬件兼容性要求严格、部署难度大



Type 1 (Bare Metal) Virtualization

硬件资源的虚拟化原理

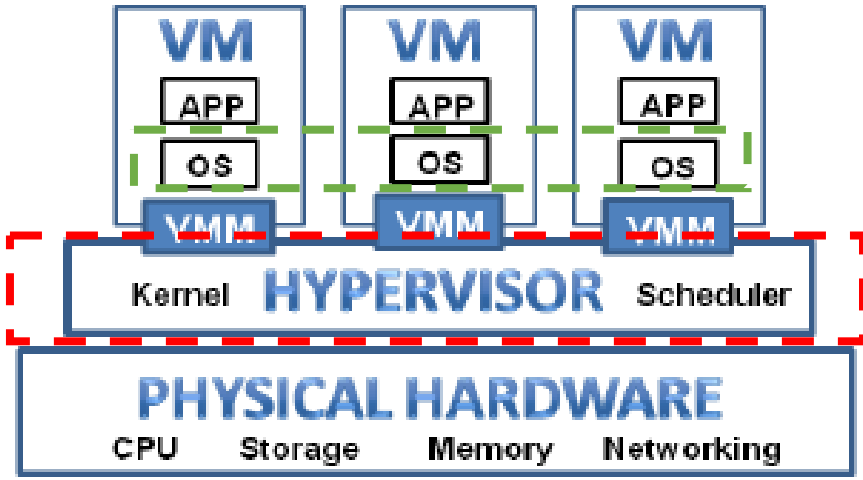
- **CPU 虚拟化**
  - VT-x 硬件加速
- **内存虚拟化**
  - 扩展页表 (EPT)
- **I/O 虚拟化**
  - 模拟—半虚拟化—直通



CPU虚拟化的原理

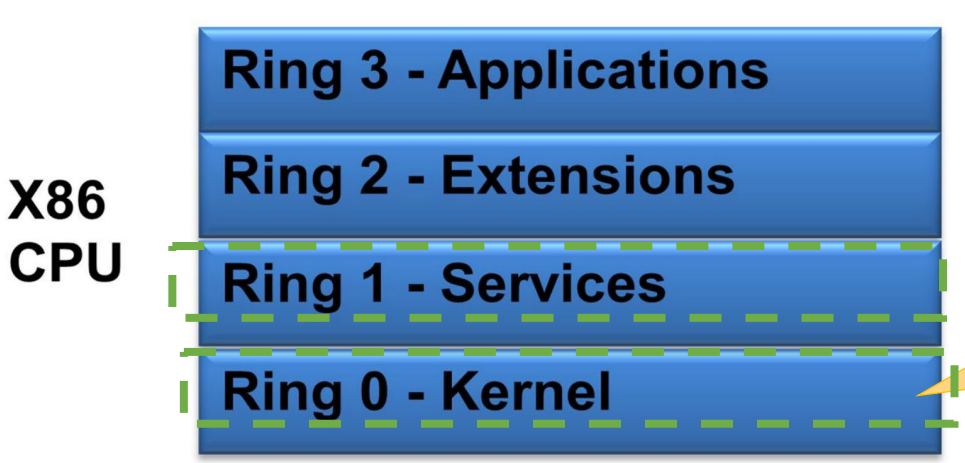
- **CPU 虚拟化**
  - VT-x: Intel X86
  - VT-i: Intel Itanium
  - AMD-v: AMD
- CPU指令分级

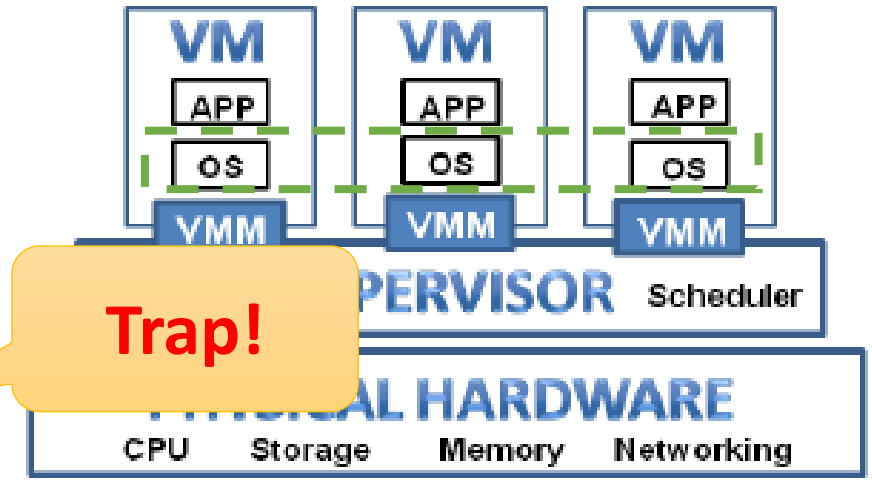




CPU虚拟化的原理

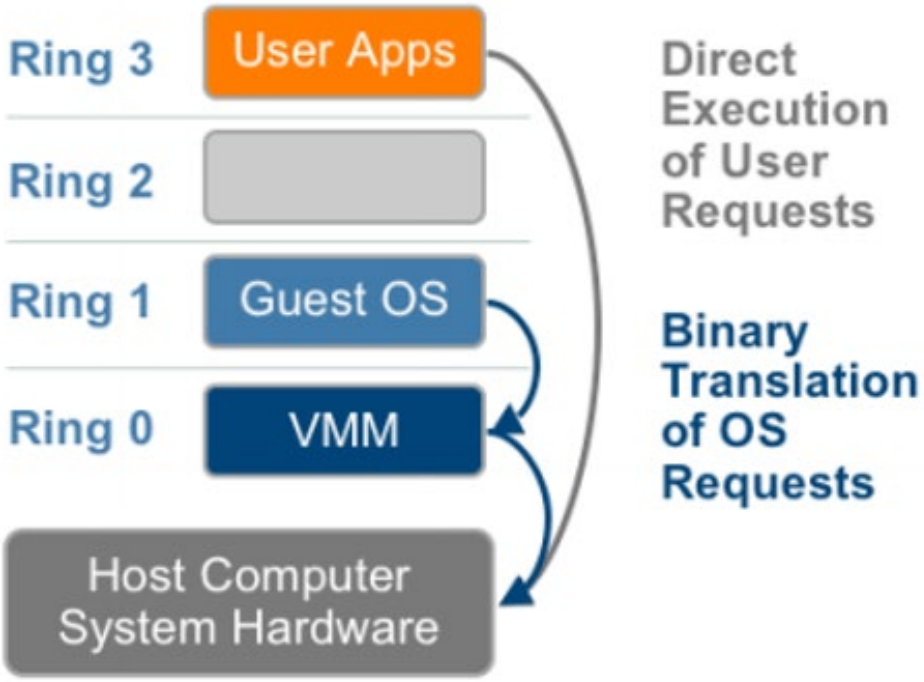
- CPU 虚拟化
  - VT-x: Intel X86
  - VT-i: Intel Itanium
  - AMD-v: AMD
- CPU指令分级





CPU虚拟化的原理

- 怎么解决CPU指令的权限冲突?
  - 软件虚拟化：VMM截获Ring0指令并转化
  - 硬件虚拟化：在CPU芯片直接支撑虚拟化指令



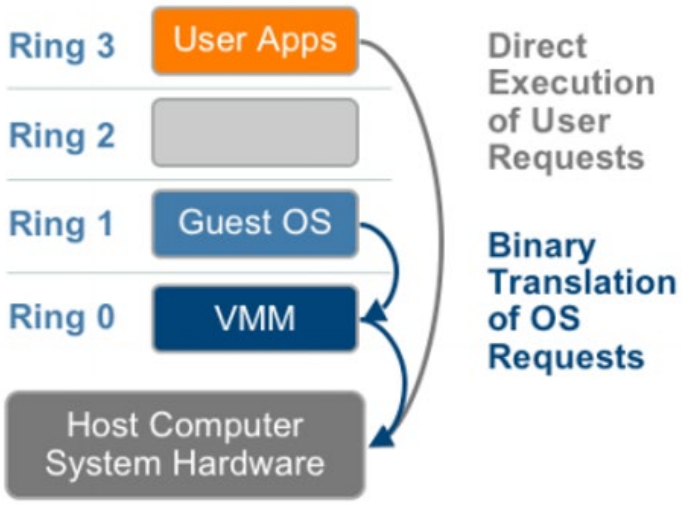
CPU虚拟化的原理：VT-x核心思想

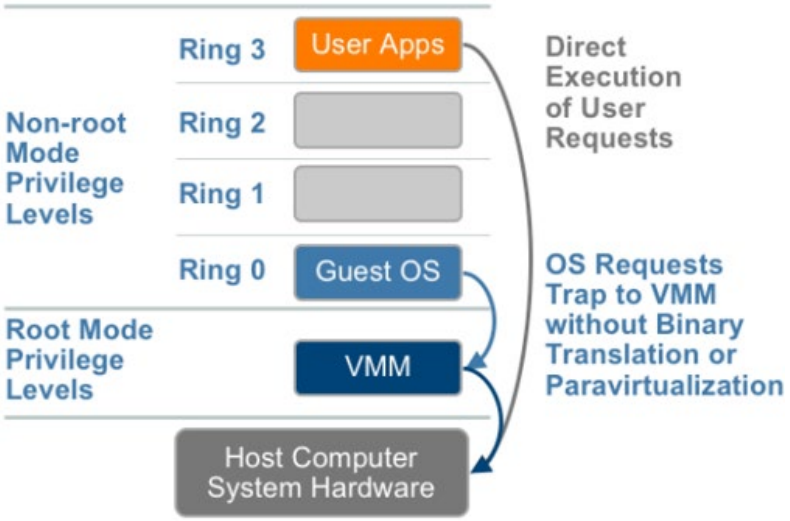
平行宇宙



CPU虚拟化的原理

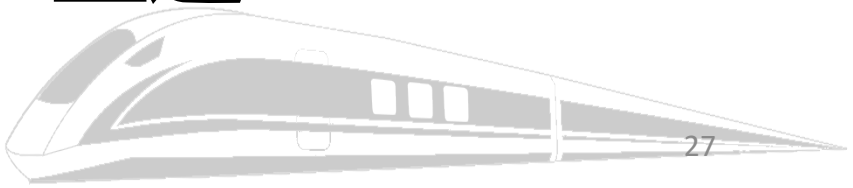
- VT-x引入了两种操作模式
  - **根操作模式（Root）**：VMM 运行所处的模式
  - **非根操作模式 （Non-Root）**：VM运行所处的模式
  - Non-Root模式也有一套Ring0-Ring3，Guest OS可以直接使用这套指令（**硬件直接运行**）
  - 只有当Guest OS试图执行影响物理机硬件的指令时，才会将控制权交给VMM转换





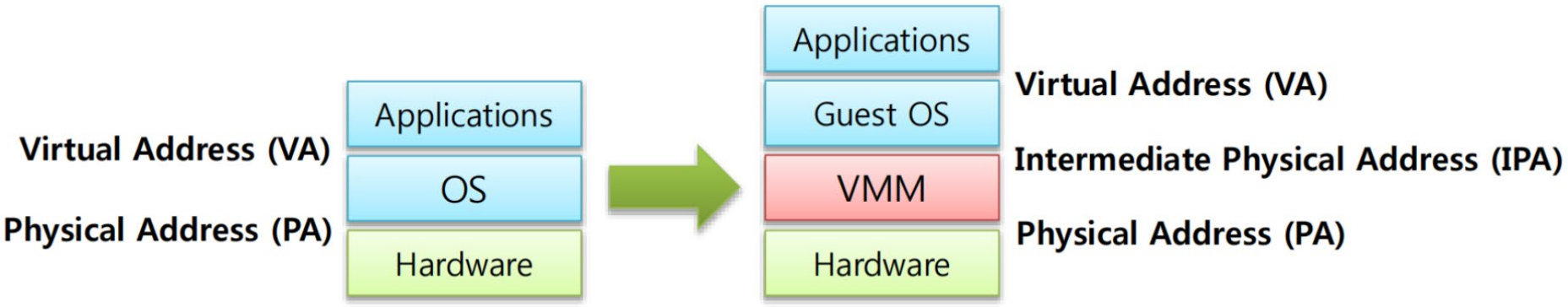
内存虚拟化的原理

- CPU 虚拟化
  - VT-x 硬件加速
- 内存虚拟化
  - 扩展页表 (EPT)
- I/O 虚拟化
  - 模拟—半虚拟化—直通



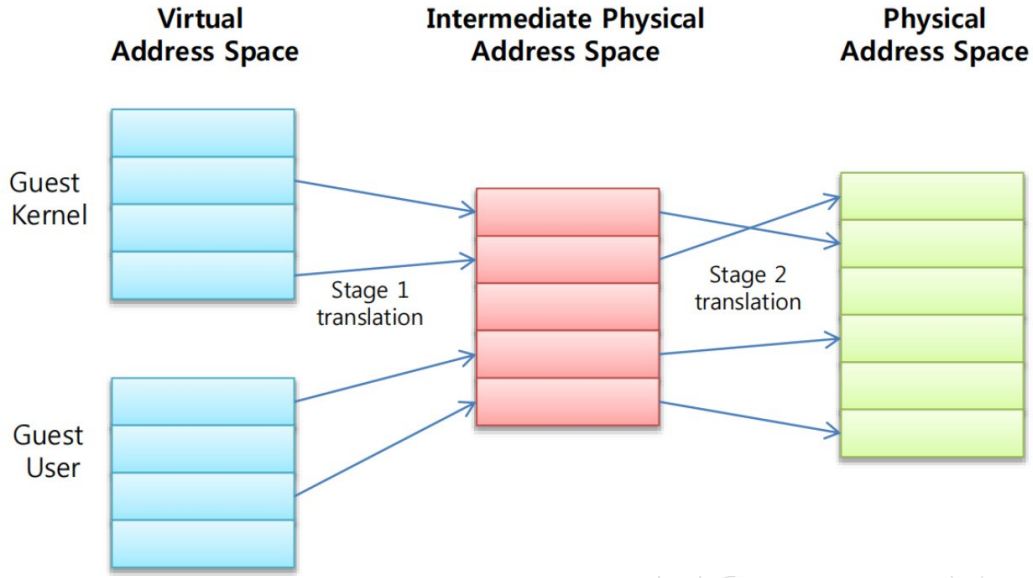
内存虚拟化的原理

- 内存地址转换：
  - 物理机：虚拟地址 → 物理地址
  - 虚拟机：虚拟地址 →Guest伪物理地址 →物理地址



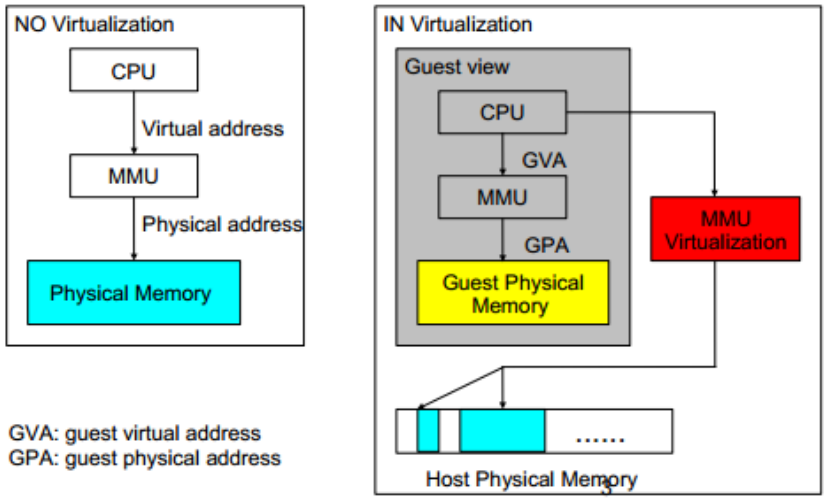
内存虚拟化的原理

- 怎么完成两次内存地址转换？
  - 影子页表（Shadow Page Table，SPT）
    - Hypervisor自己维护一个影子页表
    - 记录Guest伪物理地址到物理地址的转换
    - 拦截、更新、同步所有的内存转换
    - 传递转换后的地址给物理机CPU
  - 局限：两次转换，内存读写缓慢



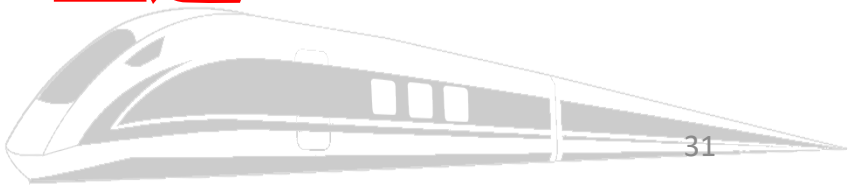
内存虚拟化的原理

- 怎么完成两次内存地址转换?
  - 软件：影子页表（Shadow Page Table，SPT）
  - 硬件：扩展页表（Extended Page Table，EPT）
    - 内存管理单元（MMU）进行两次地址转换
    - 硬件查询维护，效率高速度快



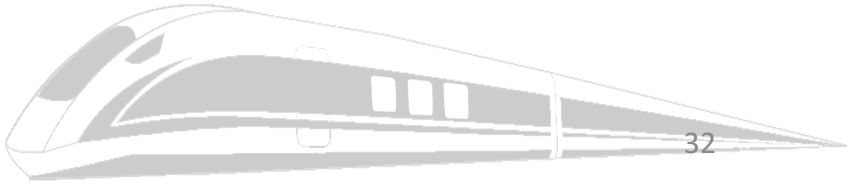
I/O虚拟化的原理

- **CPU 虚拟化**
  - VT-x 硬件加速
- **内存虚拟化**
  - 扩展页表 (EPT)
- **I/O 虚拟化**
  - 模拟—半虚拟化—直通



I/O虚拟化的原理

- 全虚拟化 (Full Virtualization)
  - VMM 纯软件截获并模拟硬件行为，兼容性最高
- 半虚拟化 (Paravirtualization)
  - Guest OS 安装特制驱动，主动与 VMM 协同完成 I/O 传输，性能更优
  - Xen
- 硬件直通 (Passthrough/SR-IOV)
  - 虚拟机直接独占物理网卡，实现零损耗

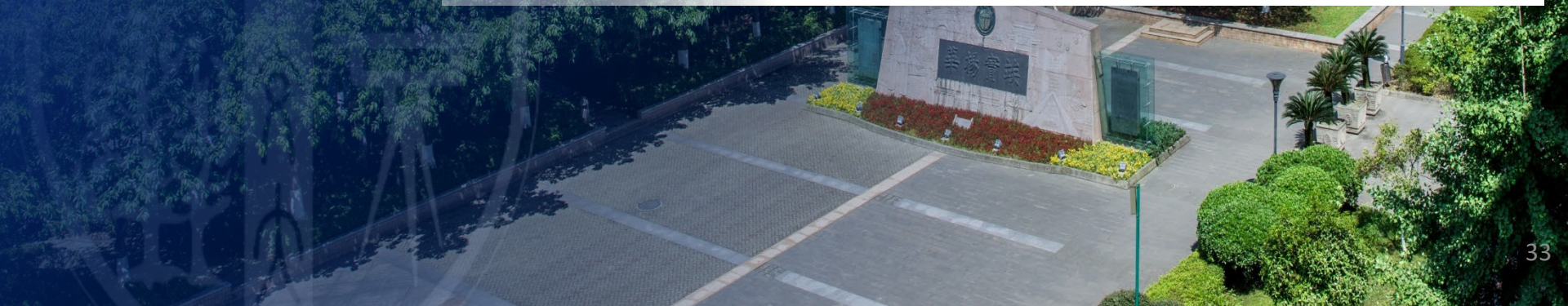


01、虚拟化技术简介

02、虚拟化技术原理

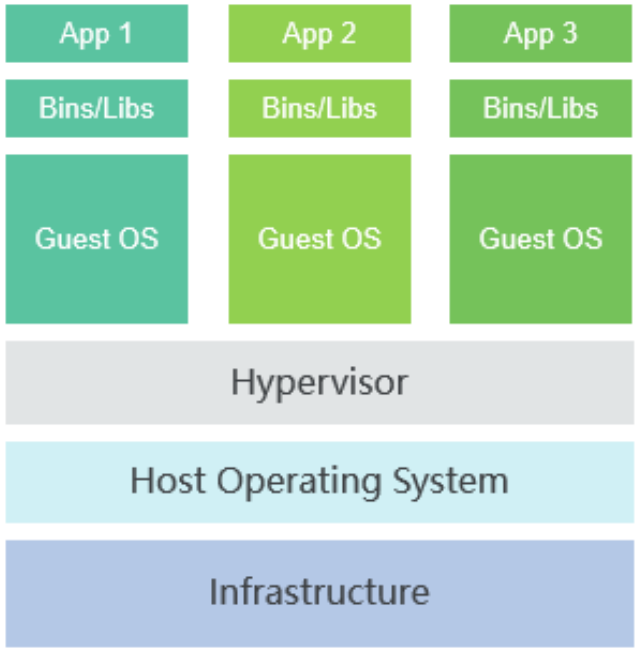
03、Docker基本原理

04、代码实践



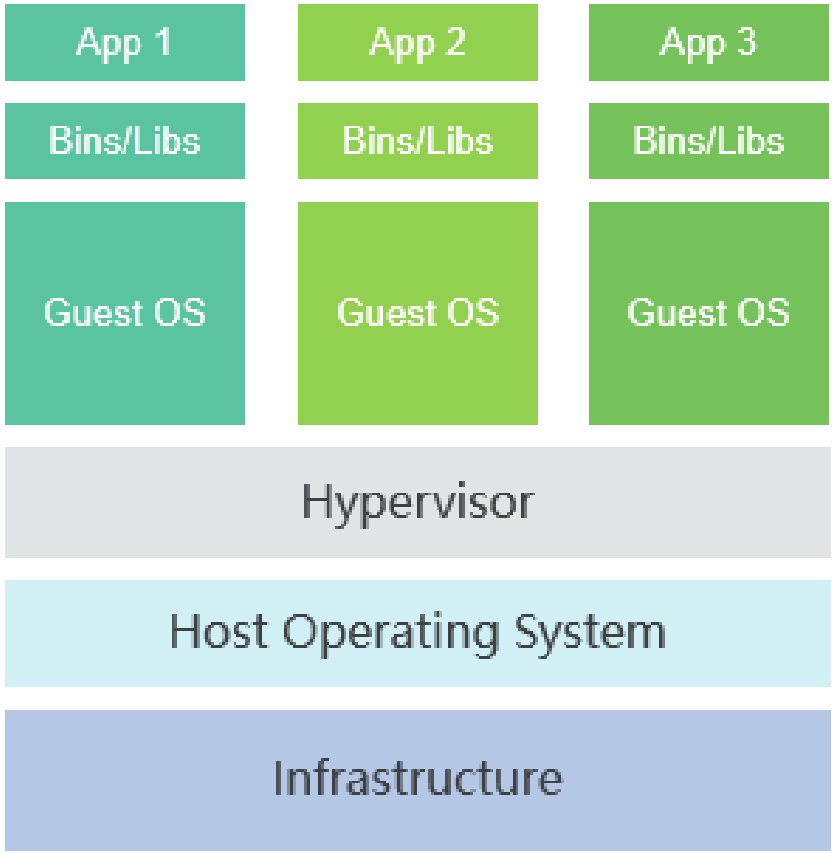
虚拟化技术的局限性

- 资源冗余：每一个虚拟机都必须运行一个完整Guest OS
- 弹性迟缓：分钟级的系统启动时间
- 交付困难：开发、测试与生产环境的底层依赖不一致问题

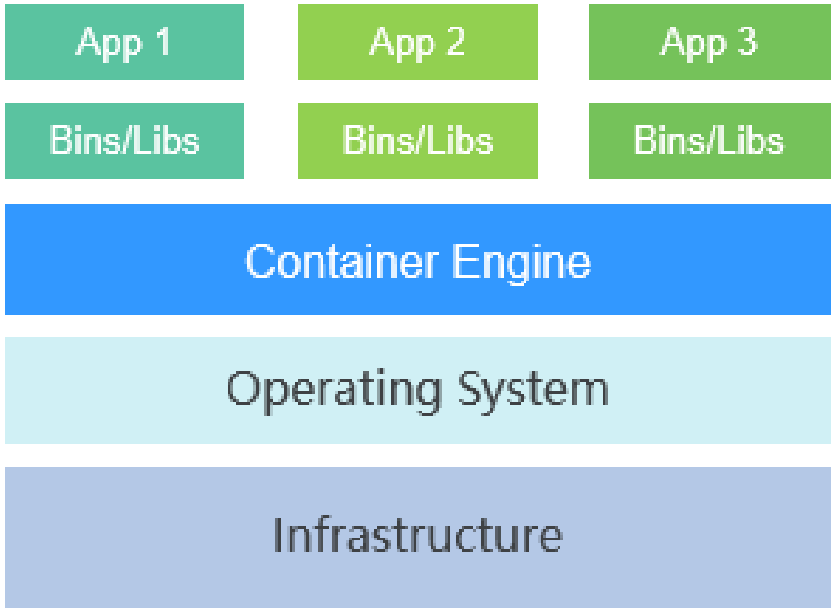


传统虚拟化

容器化技术

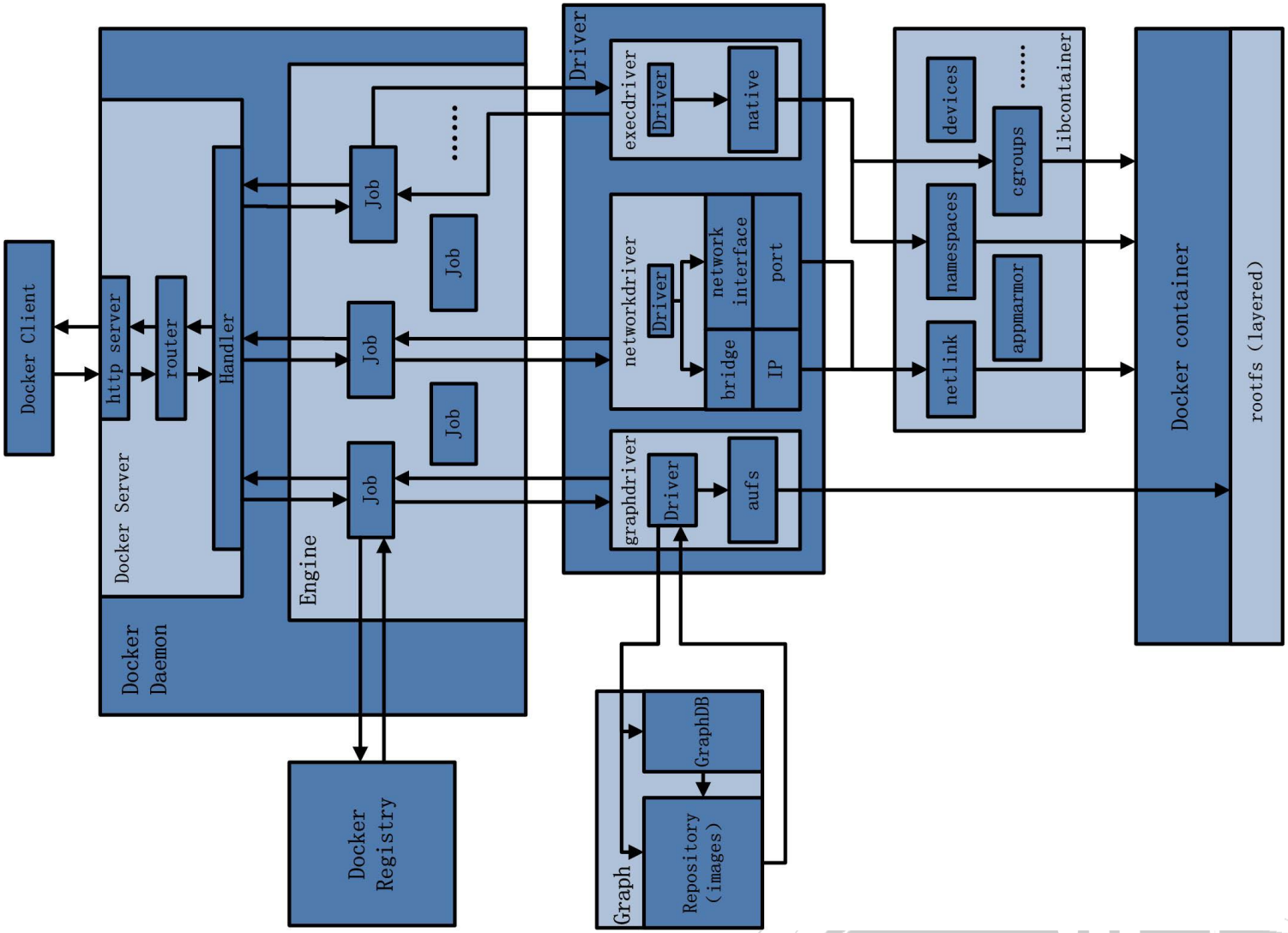


传统虚拟化

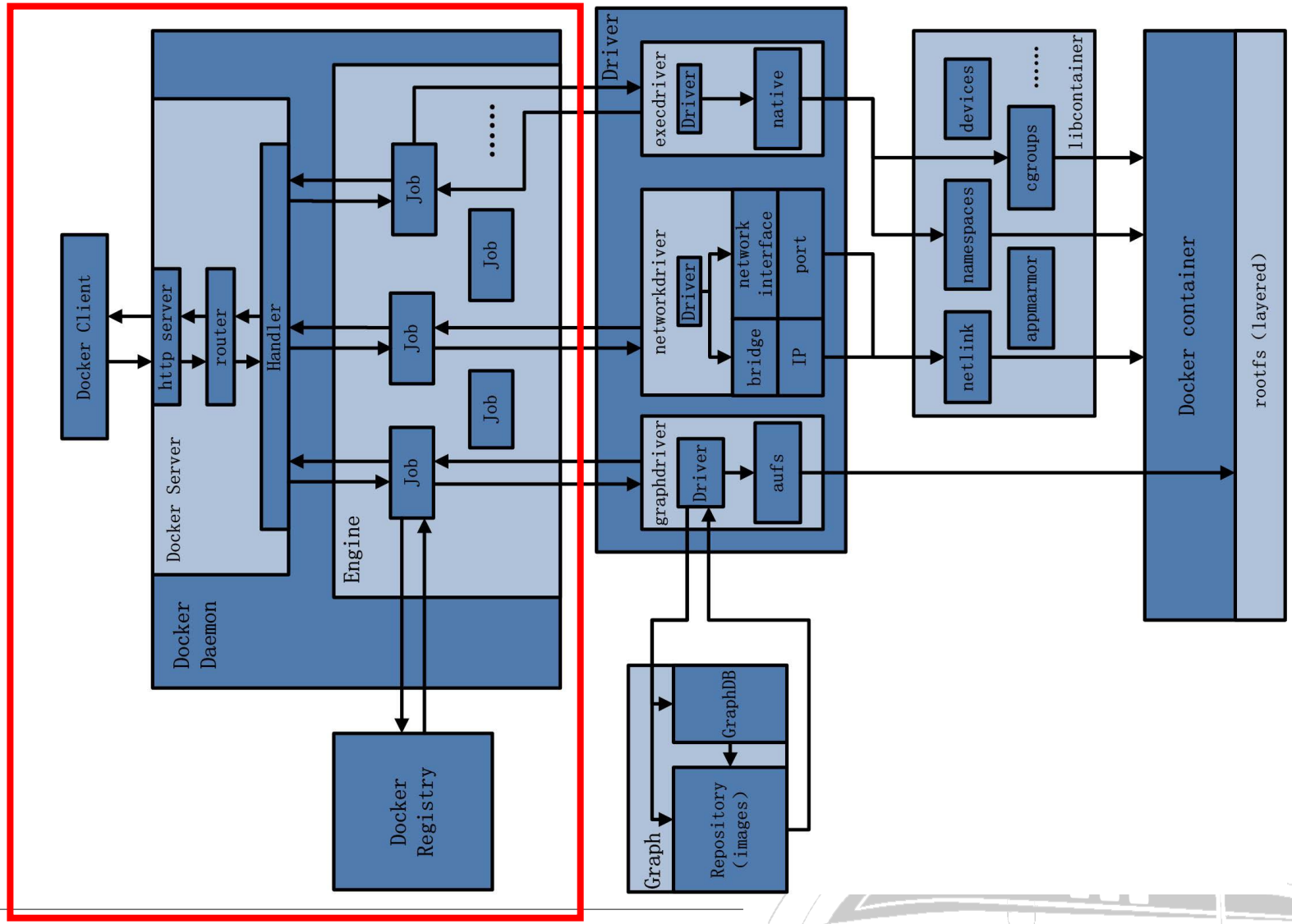


容器化

Docker架构

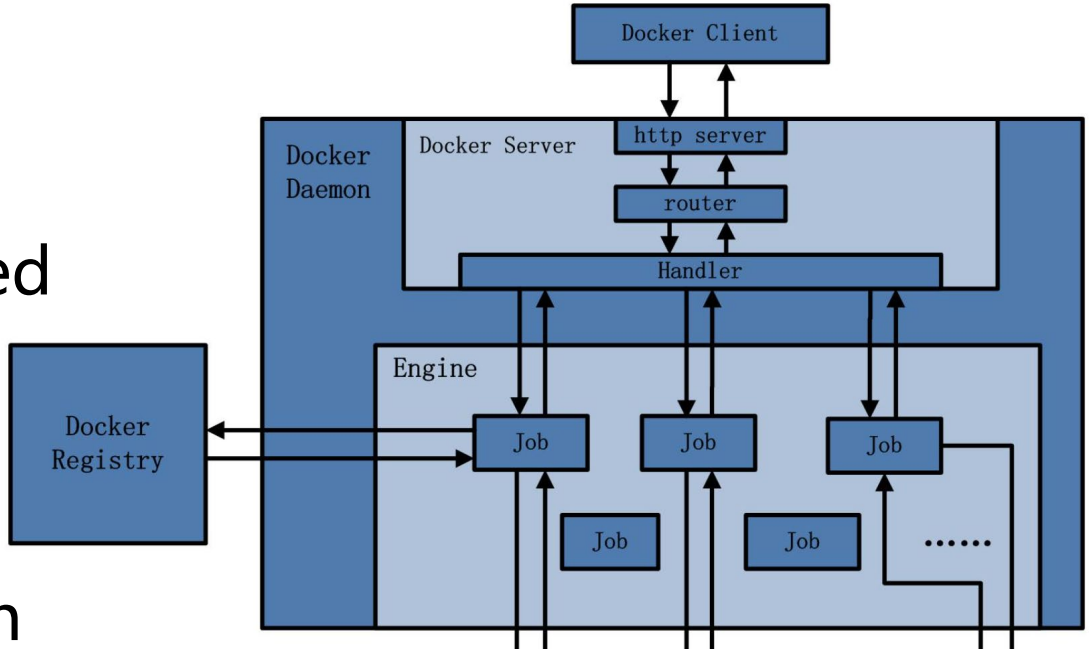

Docker Client
Docker Daemon
Docker Server
http server
router
Handler
Engine
Job
Job
Job
Job
Job
Job
Driver
execdriver
Driver
native
networkdriver
Driver
network interface
bridge
port
IP
graphdriver
Driver
aufs
libcontainer
devices
namespaces
cgroups
.....
netlink
appmarmor
Docker container
rootfs (layered)
Docker Registry
Graph
GraphDB
Repository (images)

Docker架构


Docker Client
Docker Daemon
Docker Server
http server
router
Handler
Engine
Job
Job
Job
Job
Job
Docker Registry
Graph
GraphDB
Repository (images)
Driver
execdriver
Driver
native
networkdriver
Driver
network interface
bridge
port
IP
graphdriver
Driver
aufs
libcontainer
devices
namespaces
cgroups
appmarmor
netlink
Docker container
rootfs (layered)

Docker架构

- Client/Server架构
- Docker Client
  - build, run
- Docker Daemon
  - 守护进程 dockered
- Docker Registry
  - 容器镜像仓库
  - pull, push, search

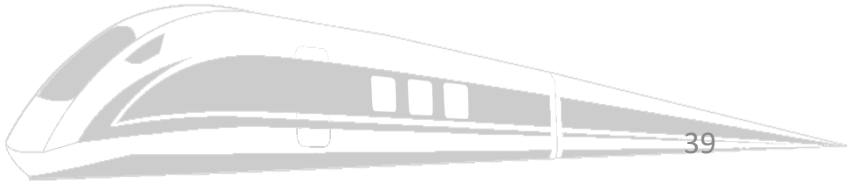


一条Docker指令会经历哪些流程?

Linux内核基石

Docker容器是怎么隔离进程的?

- 封装了Linux内核的API
  - Namespace 隔离
  - Cgroup 资源限制



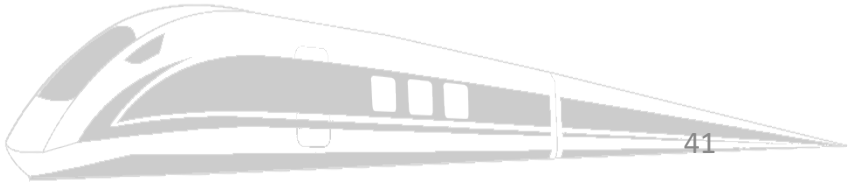
Linux内核基石：Namespace

- Linux提供的一种内核级别环境隔离的方法
- include/linux/sched.h; include/linux/nsproxy.h；kernel/nsproxy.c
  - nsproxy 是进程（task_struct）到所有 Namespace 的中转站

```
struct task_struct {
    ……..
    /* namespaces */
    struct nsproxy *nsproxy;
    …….
}
struct nsproxy {
    atomic_t count;     // nsproxy可以共享使用，count字段是该结构的引用计数
    struct uts_namespace *uts_ns;
    struct ipc_namespace *ipc_ns;
    struct mnt_namespace *mnt_ns;
    struct pid_namespace *pid_ns_for_children; // 解决进程号隔离性
    struct net             *net_ns;            // 解决网络相关设备、路由表、socket等资源的隔离性
};
```

Linux内核基石：Namespace

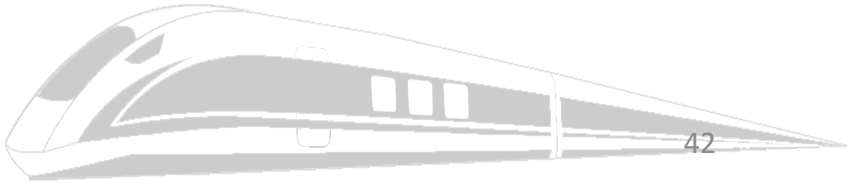
- 容器内的进程运行的时候看不到宿主机的进程
  - PID Namespace : 进程隔离
  - NET Namespace: 网络隔离
  - MNT Namespace: 挂载/文件系统隔离)
  - UTS Namespace: 主机名隔离(Ubuntu-Server)
  - IPC Namespace: 进程间通信隔离（消息队列IPC）
  - USER Namespace： 用户权限隔离



Linux内核基石：Namespace

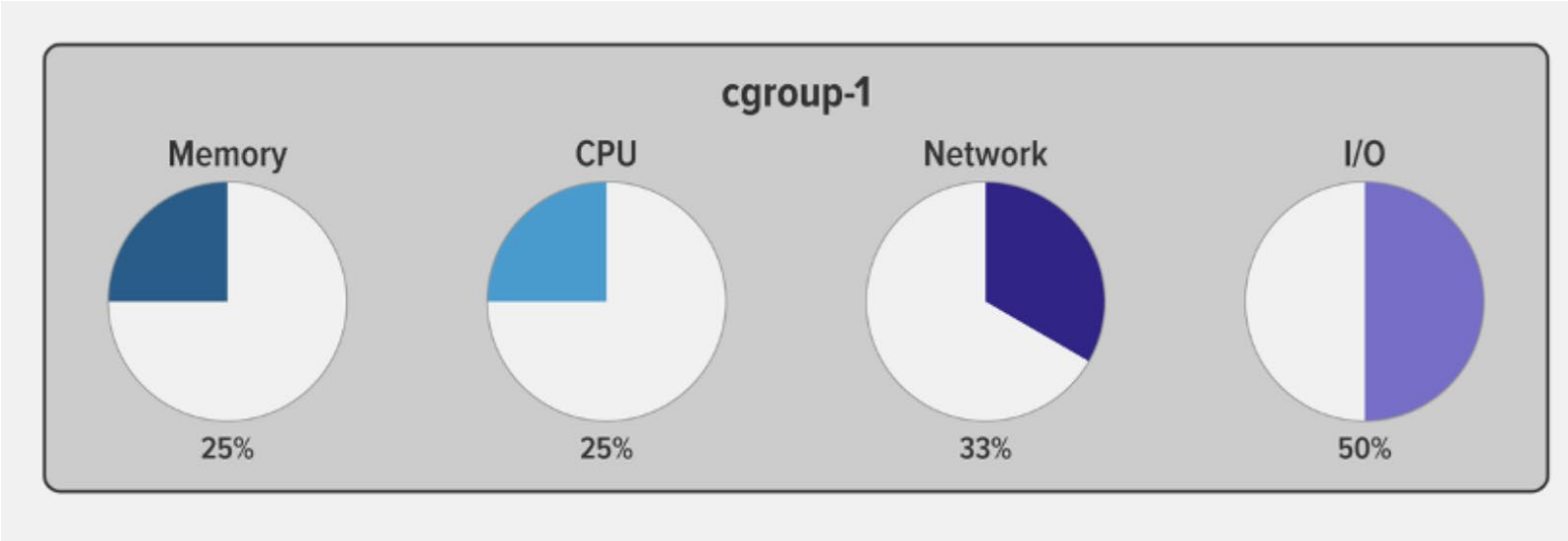
```
struct nsproxy {
    atomic_t count;
    struct uts_namespace *uts_ns;
    struct ipc_namespace *ipc_ns;
    struct mnt_namespace *mnt_ns;
    struct pid_namespace *pid_ns_for_children;
    struct net           *net_ns;
};
```

Linux: clone(CLONE_NEWPID|CLONE_NEWNET)



Linux内核基石：Cgroup

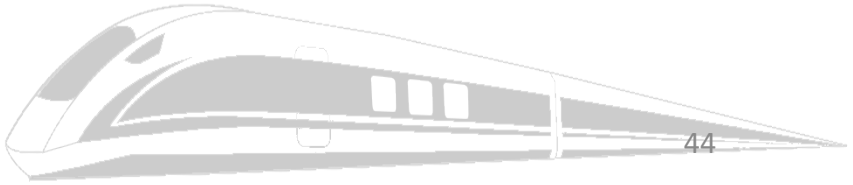
- Linux提供的一种内核级别资源限制的方法
- Cgroup：control group
- 控制什么：一组进程的资源使用、进程起始、优先级
- /sys/fs/cgroup



Linux内核基石：Cgroup

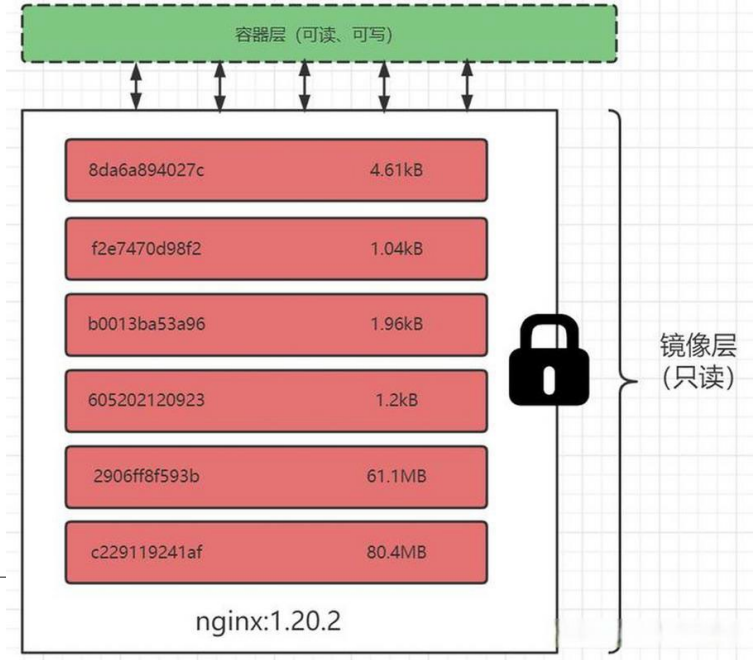
- 可控制的资源有哪些?

```
# CentOS 8.2
$ mount -t cgroup
cgroup on /sys/fs/cgroup/systemd type cgroup (rw,nosuid,nodev,noexec,relatime,xattr,release_age
cgroup on /sys/fs/cgroup/rdma type cgroup (rw,nosuid,nodev,noexec,relatime,rdma)
cgroup on /sys/fs/cgroup/freezer type cgroup (rw,nosuid,nodev,noexec,relatime,freezer)
cgroup on /sys/fs/cgroup/cpu,cpuacct type cgroup (rw,nosuid,nodev,noexec,relatime,cpu,cpuacct)
cgroup on /sys/fs/cgroup/net_cls,net_prio type cgroup (rw,nosuid,nodev,noexec,relatime,net_cls,
cgroup on /sys/fs/cgroup/blkio type cgroup (rw,nosuid,nodev,noexec,relatime,blkio)
cgroup on /sys/fs/cgroup/memory type cgroup (rw,nosuid,nodev,noexec,relatime,memory)
cgroup on /sys/fs/cgroup/devices type cgroup (rw,nosuid,nodev,noexec,relatime,devices)
cgroup on /sys/fs/cgroup/pids type cgroup (rw,nosuid,nodev,noexec,relatime,pids)
cgroup on /sys/fs/cgroup/cpuset type cgroup (rw,nosuid,nodev,noexec,relatime,cpuset)
cgroup on /sys/fs/cgroup/hugetlb type cgroup (rw,nosuid,nodev,noexec,relatime,hugetlb)
cgroup on /sys/fs/cgroup/perf_event type cgroup (rw,nosuid,nodev,noexec,relatime,perf_event)
```



镜像与容器的分层结构

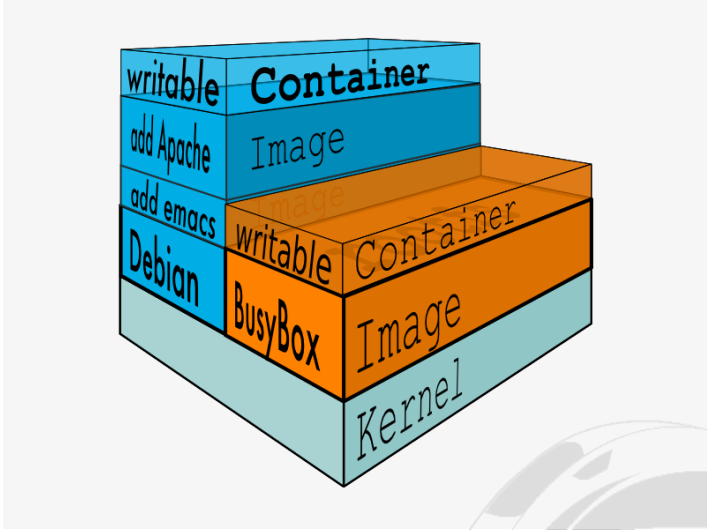
- 镜像层、容器层
  - 容器运行时会在镜像层只读层基础上添加可写层
  - 相同基础层（系统环境、语言库...）服务器仅存一次
  - 大规模镜像分发：网络和存储成本大幅降低



联合文件系统（UnionFS）

镜像层与容器层怎么动静结合联合使用?

- 将分属不同物理位置的只读目录合并，挂载至一个虚拟文件系统位置（挂载点），实际物理位置是分开的
- 容器启动后, Docker Engine在最顶端动态附加一层可写层
- 容器生命周期内，所有文件系统操作均发生于顶部可写层

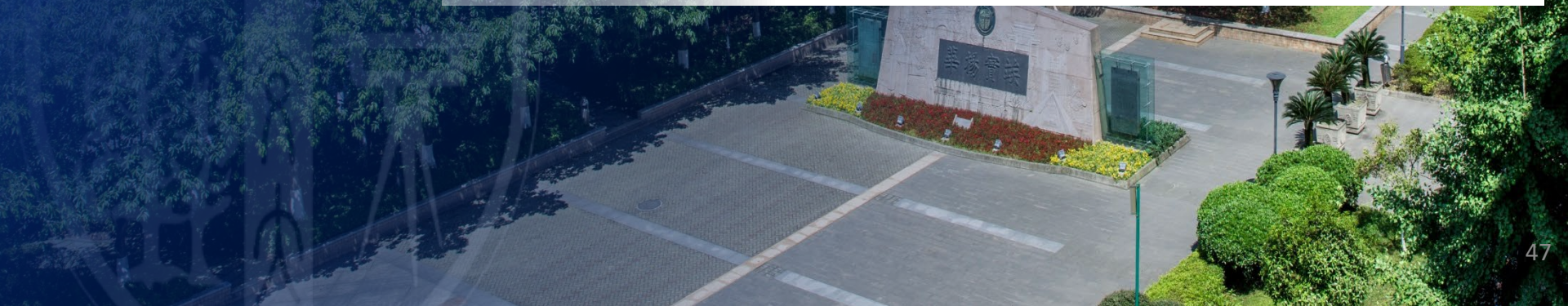


01、虚拟化技术简介

02、虚拟化技术原理

03、Docker基本原理

04、代码实践

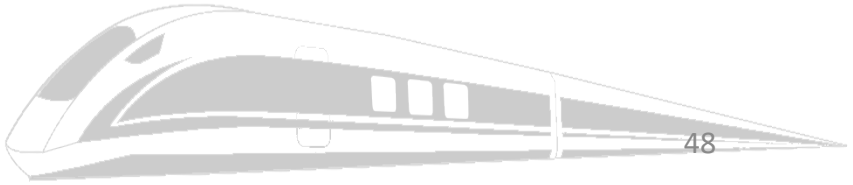


基于 Python SDK 的容器生命周期管理

- 建立连接
- 拉取镜像

```
import docker


# 1. 建立与宿主机本地 Docker 守护进程 (Daemon) 的套接字连接
client = docker.from_env()
# 2. 从云端 Registry (如 Docker Hub) 自动化拉取指定镜像
print("📥 正在通过 API 拉取 nginx:alpine 轻量级镜像...")
image = client.images.pull("nginx", tag="alpine")
print(f"✅ 镜像拉取成功! 占用存储: {image.attrs['Size'] / 1024 / 1024:.1f} MB")
```



基于 Python SDK 的容器生命周期管理

- 创建容器
- 启动容器

```
# 创建并运行容器
print("\n🚀 创建并启动 Nginx 容器...")
container = client.containers.run(
    "nginx:alpine",                    # 使用的镜像
    name="demo-nginx",                 # 容器名称
    ports={'80/tcp': 8080},            # 端口映射: 容器80 → 宿主机8080
    detach=True,                       # 后台运行
    environment={                      # 环境变量
        "NGINX_HOST": "localhost",
        "NGINX_PORT": "80"
    },
    labels={                           # 容器标签
        "course": "cloud-computing",
        "lecture": "02-docker"
    }
)

# 等待容器完全启动
time.sleep(2)
container.reload()  # 刷新容器状态
```

基于 Python SDK 的容器生命周期管理

- 查看状态
- 停止删除

```
# 4. 状态监控：获取容器内部的标准输出流 (Logs)
logs = container.logs(tail=5).decode('utf-8')
print(f"📝 容器实时运行日志: \n{logs}")

# 5. 资源回收：阻断进程并强制销毁实例环境
container.stop(timeout=2)
container.remove()

print("🗑️ 容器实例已安全销毁，计算与网络资源瞬间完全释放！")
```




西南交通大學
Southwest Jiaotong University
THANKS
丛荣
西南交通大学
计算机与人工智能学院